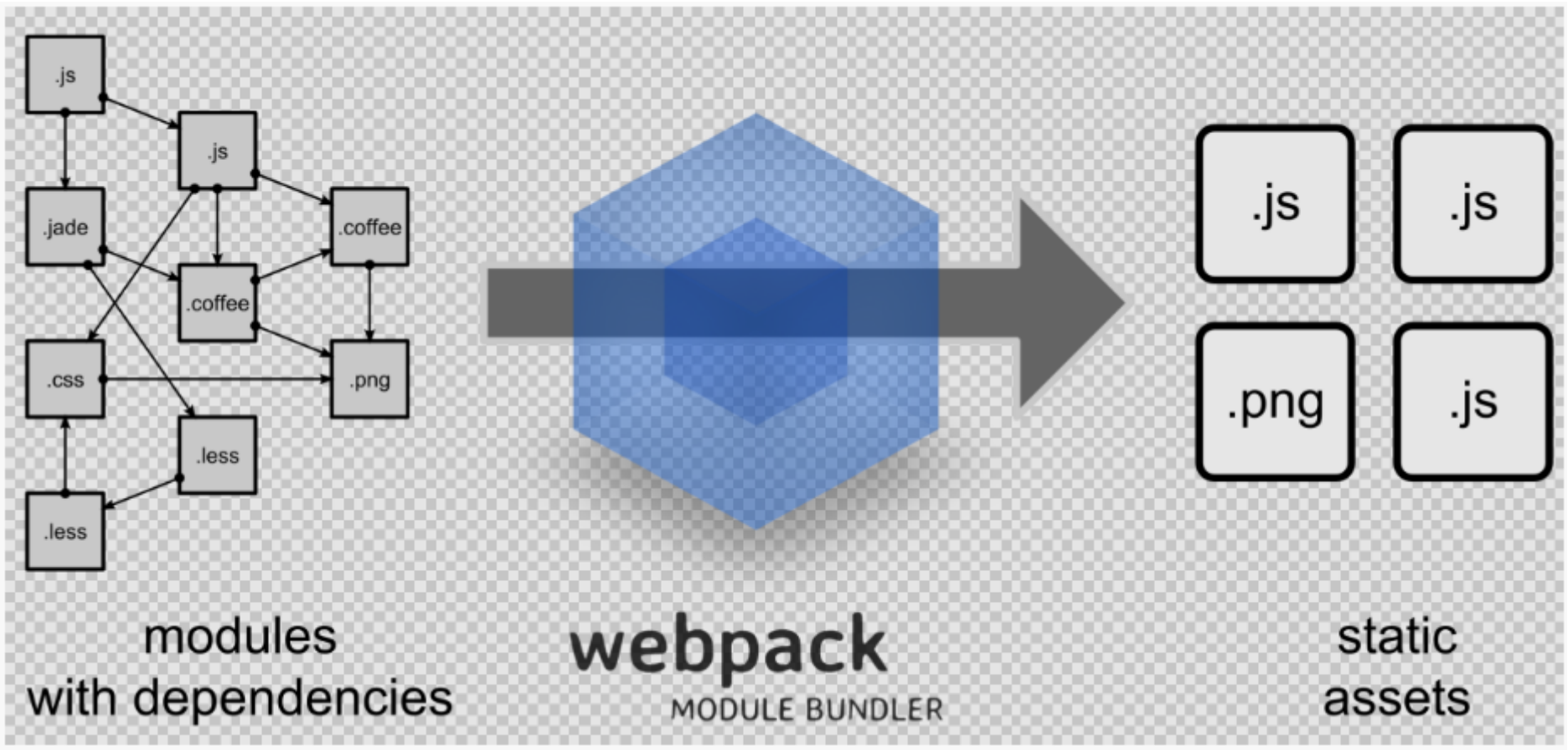
WebPack详解

简介

Webpack 是一个前端资源加载/打包工具。它将根据模块的依赖关系进行静态分析，然后将这些模块按照指定的规则生成对应的静态资源。

从图中我们可以看出，**Webpack** 可以将多种静态资源 js、css、less 转换成一个静态文件，减少了页面的请求。



安装

- 全局安装

```
npm install -g webpack webpack-cli
```

- 安装后查看版本号

```
webpack -v
```

初始化项目

- 创建**webpack**文件夹

```
npm init- y
```

- 创建src文件夹
- src文件夹下创建common.js

```
exports.info = function(str){
    document.write(str);
}
```

- src文件夹下创建utils.js

```
exports.add = function(a,b){
    return a + b;
}
```

- src文件夹下创建main.js

```
const common = require("./common")
const utils = require("./utils")

common.info("Hello World!" + utils.add(1,2))
```

JS打包

- **webpack**目录下创建配置文件**webpack.config.js**

```
const path = require("path") // NodeJS内置模块
module.exports = {
    entry:'./src/main.js', // 配置入口文件
    output:{
        // 输出路径，__dirname：当前文件所在路径
        path:path.resolve(__dirname,'./dist'),
        //输出文件
        filename:'bundle.js'
    }
}
```

以上配置的意思是：读取当前项目目录下 `src` 文件夹中的 `main.js` （入口文件）内容，分析资源依赖，把相关的 `js` 文件打包，打包后的文件放入当前目录的dist文件夹下，打包后的 `js` 文件名为 `bundle.js`

- 命令行执行编译命令

1. 执行命令

```
webpack --mode=development
```

执行后查看 `bundle.js` 里面包含了上面两个 `js` 文件的内容并进行了代码压缩

2. 也可以配置项目的 `npm` 运行命令，修改 `package.json` 文件

```
"scripts": {
    //...,
    "dev": "webpack --mode=development"
}
```

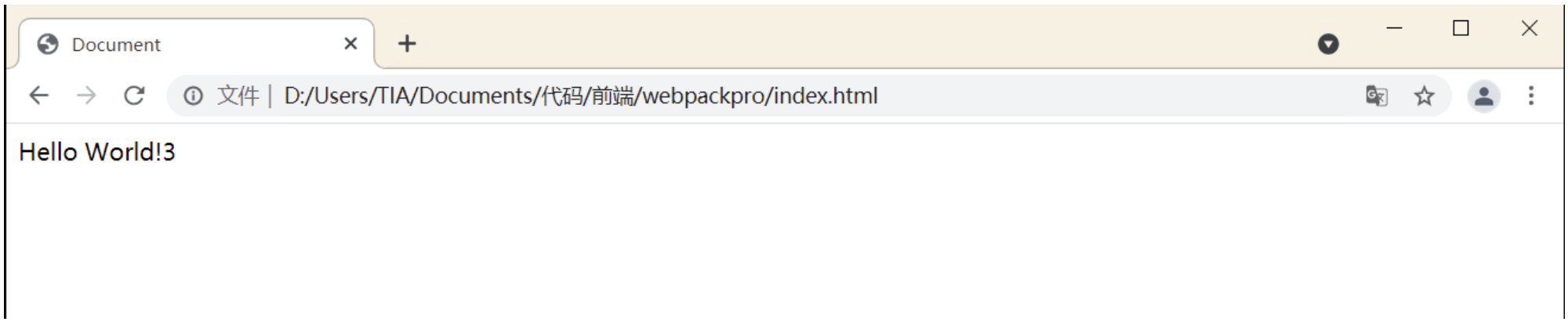
运行npm命令执行打包

```
npm run dev
```

- webpack目录下创建 `index.html` ，引用 `bundle.js`

```
<body>
    <script src="./dist/bundle.js"></script>
</body>
```

- 浏览器中查看 `index.html`



CSS打包

- 安装**style-loader**和 **css-loader**

Webpack 本身只能处理 JavaScript 模块，如果要处理其他类型的文件，就需要使用 loader 进行转换。

Loader 可以理解为是模块和资源的转换器。

首先我们需要安装相关Loader插件

- **css-loader 是将 css 装载到 javascript**
- **style-loader 是让 javascript 认识css**

1. 安装

```
npm install --save-dev style-loader css-loader
```

2. 修改webpack.config.js

```
const path = require("path"); //Node.js内置模块
module.exports = {
    //...,
    output:{
        //其他配置
    },
    module: {
        rules: [
            {
                test: /\.css$/,    //打包规则应用到以css结尾的文件上
```

```
                use: ['style-loader', 'css-loader']
            }
        ]
    }
}
```

3. **在src文件夹** 创建style.css

```
body{
    background:pink;
}
```

- 修改main.js，在第一行引入style.css

```
require('./style.css');
```

- 运行编译命令

```
npm run dev
```

- 浏览器中查看 **index.html** ， 查看背景颜色变化